SRINIVAS UNIVERSITY

INSTITUTE OF ENGINEERING & TECHNOLOGY
MUKKA,MANGALURU - 574146

DEPARTMENT OF ARTIFICIAL INTELLIGENCE AND MACHINE LEARNING ENGINEERING

MINI REPORT
ON

" PRACTICAL APPLICATIONS OF BIG DATA, MACHINE LEARNING, NEURAL NETWORKS, AND GENERATIVE AI "

Submitted in the partial fulfillment of the requirements for the award of the degree of

BACHELOR OF TECHNOLOGY
IN
ARTIFICIAL INTELLIGENCE AND MACHINE LEARNING ENGINEERING

Submitted By,

ANUSHA SANKANAGOUDAR	**01SU25AI014**
FATHIMA RAMEEZA	**01SU25AI027**
KHATHEEJA AFRA	**01SU25AI035**
MAHAMMED FAYAZUDDEEN	**01SU25AI040**

UNDER THE GUIDANCE OF

PROF. MAHESH KUMAR VB

DEPARTMENT OF ARTIFICIAL INTELLIGENCE AND MACHINE LEARNING ENGINEERING
2025-2026

CONTENTS

Module 2 BIG DATA PROCESS MAPPING **2**

2.1 Introduction and Problem Statement 2

2.2 Data Sources 2

2.3 Data Ingestion and Storage 3

2.4 Processing Layer 3

2.5 Output Layer and Feedback Loop 3

2.6 The Five Vs Applied to the Recommendation System 3

Module 3 BUILD A SIMPLE ML PROCESS FLOW **4**

3.1 Introduction and Problem Definition 4

3.2 Data Collection and Preparation 4

3.3 Feature Extraction and Algorithm Selection 5

3.4 Training and Validation 5

3.5 Testing, Evaluation and Deployment 5

Module 4 BACKPROPAGATION WALKTHROUGH **6**

4.1 Introduction and Network Setup 6

4.2 Forward Pass and Loss 6

4.3 Backward Pass and Gradient Flow 6

4.4 Weight Update and Verification 7

4.5 Observations 7

Module 5 CONTENT AI EVALUATION CHALLENGE **8**

5.1 Introduction and Objective 8

5.2 Content Generation Using an NLP Tool 8

5.3 Evaluating Clarity and Tone 8

5.4 Evaluating Body-Language Cues in Multimedia 9

5.5 Findings, Ethics and Conclusion 9

5.6 Overall Conclusion and Future Scope 10

5.7 References 11

MODULE 2
BIG DATA PROCESS MAPPING

Question (Module 2): *Big Data Process Mapping: Groups select a real-world big data system (like Google Maps, Amazon recommendations, or smart city sensors) and map out the entire data flow: data sources, storage, processing, and output.*

2.1 Introduction and Problem Statement

Big data is the term used for collections of information that are so large, so fast-moving or so varied that a single computer and a conventional database cannot capture, store and analyse them in a useful amount of time. It is usually described through the five Vs: **volume**, the sheer quantity of records; **velocity**, the speed at which new records arrive; **variety**, the mixture of tables, text, images and event logs; **veracity**, the degree to which the data can be trusted; and **value**, the practical insight that justifies the cost of keeping and processing it. Mapping a big data process therefore means following a piece of data from the instant it is generated to the moment it influences a decision that a person actually sees, and naming every system it passes through on the way.

For this module the group selected the product recommendation system of a large online retailer, of the kind used by Amazon, as its real-world case study. Every visit to an online store produces a stream of tiny events: a page is opened, a search is typed, an item is added to a basket, a purchase is completed or a return is requested. The problem statement is simple to say but hard to solve: given hundreds of millions of shoppers and a catalogue of hundreds of millions of products, how can the platform decide, within a fraction of a second, which handful of items to show to each person? The data flow was organised under four headings: data sources, ingestion and storage, processing, and output. Figure 2.1 is a simplified, educational model built from publicly available descriptions, because the internal architecture of a commercial product is proprietary, so it should be read as a representative pipeline and not as an official blueprint.

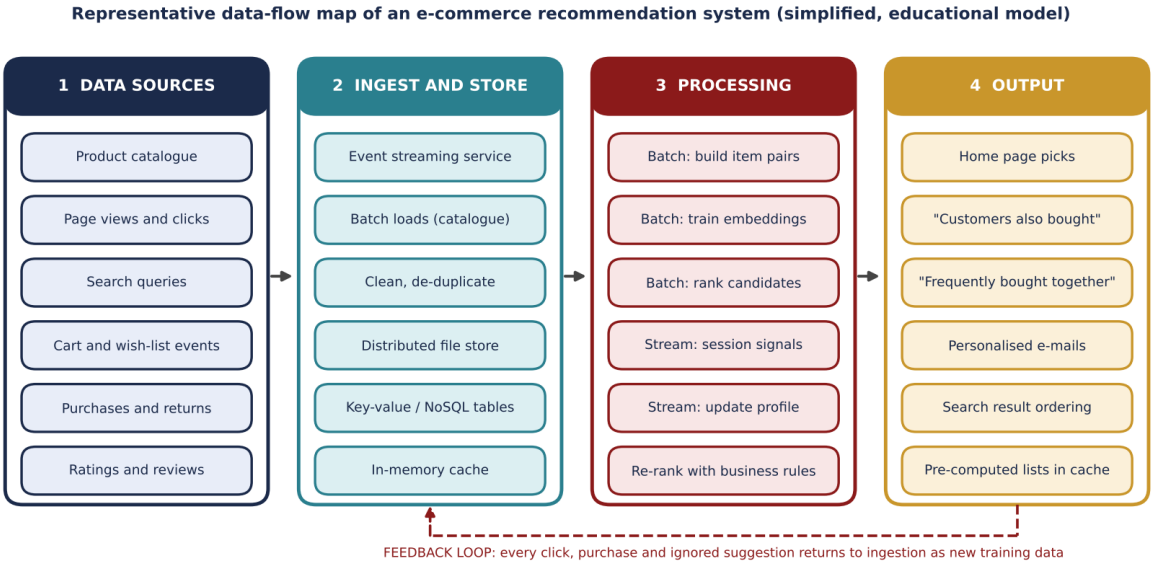


Figure 2.1: *End-to-end data-flow map of the case study, from data sources to user-facing output, with the feedback loop that returns every output to the system as new input.*

2.2 Data Sources

The first layer contains the places where the data originate, and in a recommendation system they fall into four families. Catalogue data describe the products themselves: titles, descriptions, categories, prices, images and seller details. Behavioural data record what shoppers do: page views, clicks, searches, items added to a basket or wish list, purchases, returns and the time spent on a page. Explicit feedback consists of star ratings, written reviews and customer questions and answers, while contextual data such as the device in use, the time of day, the language and the broad region help the system interpret each event.

The most valuable source is implicit behaviour, because it is produced in enormous quantities without asking anybody to do extra work. Each event is a small record holding a pseudonymous user identifier, a product identifier, an event type and a timestamp, and billions of such records reveal which products are viewed together, bought together or abandoned.

2.3 Data Ingestion and Storage

The ingestion stage must receive data reliably, however it arrives. Slowly changing data such as the catalogue and the historical order tables are loaded in large batches on a schedule, whereas the clickstream is received as a continuous stream through a distributed messaging platform of the Apache Kafka type. Such a platform absorbs sudden bursts of traffic, for example during a festival sale, and allows several consumers to read the same stream independently. Records are validated at this point: traffic generated by automated bots is filtered out, duplicate events are dropped, and personal identifiers are replaced with pseudonymous keys so that later stages never need to see a name or an address.

The storage layer then keeps each kind of data in the form that suits how it will be read. Raw logs, images and the complete history are written to a distributed file system or object store, in which every file is split into blocks that are copied across many machines so that the failure of one machine loses nothing; this raw collection is often called a data lake, and it allows old data to be reprocessed whenever a better algorithm is developed. Structured records such as product attributes, order history and user profiles are kept in horizontally scalable NoSQL or key-value databases that can fetch one record by its identifier in a few milliseconds. The hottest data, such as the current best sellers and the recent activity of each active shopper, are held in an in-memory cache.

2.4 Processing Layer

Processing turns raw records into knowledge and runs in two cooperating modes. **Batch processing** periodically scans the entire stored history on a cluster using frameworks such as MapReduce or Apache Spark. Its main task is to build an item-to-item similarity table: for every product it finds the other products that tend to be bought or viewed by the same customers. A small example makes the idea concrete. If 1,000 customers bought a camera and 300 of them also bought a memory card, while only 2 per cent of all customers buy memory cards, the lift of the pair is 0.30 divided by 0.02, which equals 15, so the memory card is an excellent candidate to recommend with the camera.

Stream processing runs continuously over the live events. It updates the session context, such as the last few products a shopper browsed, refreshes the user profile and adjusts popularity counters within seconds. When a page must be built, the system follows a two-step pattern called retrieve and rank. Retrieval pulls a few hundred candidate products from the similarity tables and embeddings, and a ranking model, typically gradient-boosted trees or a neural network, scores each candidate by its predicted probability of a click or a purchase. A final re-ranking step applies business rules: products that are out of stock or already bought are removed, the list is diversified across categories, and regional or age restrictions are respected.

2.5 Output Layer and Feedback Loop

The output layer is everything the shopper sees and receives: the personalised picks on the home page, the “customers who bought this also bought” and “frequently bought together” panels on a product page, suggestions in the cart, ordering of search results, and personalised e-mails and notifications. These outputs reach web browsers, mobile applications and e-mail systems through application programming interfaces and must appear in well under a second, so many lists are pre-computed and served from caches. Controlled A/B experiments compare click-through and conversion rates of competing models, and only the better model is promoted.

The process map is a loop rather than a straight line. Every impression, click and purchase returns to the ingestion stage as new data, and even an ignored suggestion is informative, so each output becomes an input, a property often described as the data network effect. The main challenges noted by the group were privacy and consent, which require anonymisation and compliance with laws such as the Digital Personal Data Protection Act, 2023 in India; the cold-start problem for new users and new products that have no history; popularity bias that can trap shoppers in a narrow filter bubble; and the cost of running such a system.

2.6 The Five Vs Applied to the Recommendation System

Table 2.1: The five Vs mapped to the case study

V	Meaning	Where it appears in the case study
Volume	Quantity of records	Hundreds of millions of products; billions of daily events
Velocity	Speed of arrival	Clicks and purchases stream in, with sharp sale-day peaks
Variety	Mixture of formats	Text, images, prices, reviews, clickstream logs, orders
Veracity	Trustworthiness of noisy data	Bots, fake reviews and duplicate events are filtered out
Value	Insight that justifies the cost	Relevant suggestions raise discovery and conversion

MODULE 3
BUILD A SIMPLE ML PROCESS FLOW

Question (Module 3): *Build a Simple ML Process Flow: Groups create a complete flowchart for a machine learning project, covering data collection, feature extraction, algorithm selection, training, testing, and evaluation.*

3.1 Introduction and Problem Definition

Machine learning is the branch of artificial intelligence in which programs improve at a task by learning patterns from data instead of following rules written by hand. A machine learning project is therefore not a single act of "training a model" but a disciplined sequence of stages, in which each stage produces an output that feeds the next and in which several decision points can send the team back to an earlier stage when the results disappoint. The group's task was to draw this sequence as a complete flowchart covering data collection, feature extraction, algorithm selection, training, testing and evaluation, and to explain the purpose of every symbol used in it.

To keep the flowchart concrete, a familiar supervised-learning problem was used as the running example: classifying an incoming e-mail as spam or not spam. The input is the text and header information of the message, and the output is a yes-or-no label, so the task is binary classification. The chart in Figure 3.1 begins with a rounded terminator and a process box that defines the problem and its success criteria, for instance catching at least 95 per cent of spam while wrongly blocking no more than 2 per cent of genuine mail. Rectangles stand for processing steps, parallelograms for data input and output, diamonds for yes-or-no decisions, and dashed arrows for iteration loops that return to an earlier stage.

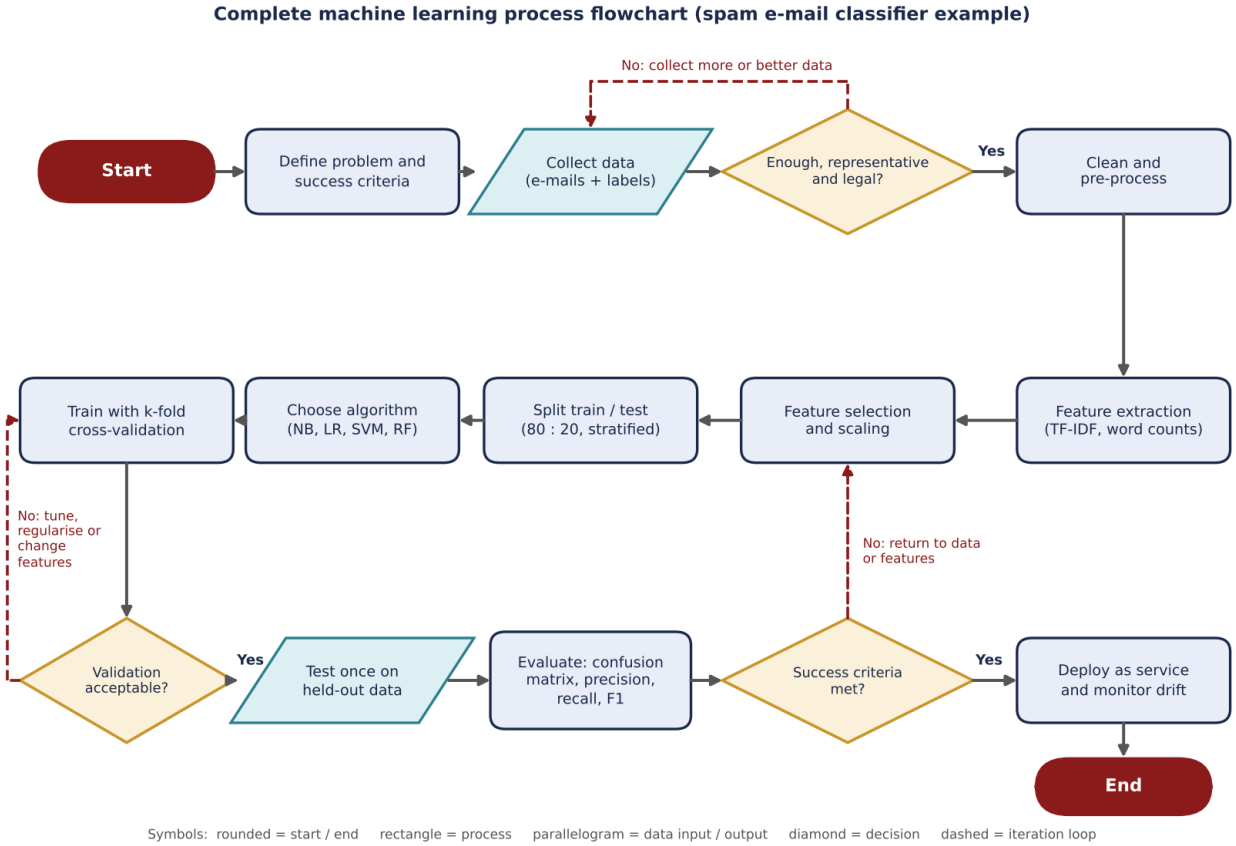


Figure 3.1: *Complete machine learning process flowchart for the spam-classification example. Dashed arrows are the iteration loops that return to earlier stages.*

3.2 Data Collection and Preparation

The first working stage is data collection. For the spam example the team gathers a labelled corpus of e-mails, such as the public SpamAssassin or Enron-Spam collections, in which each row holds one message and a label saying whether it was spam. At this point the first decision diamond asks three questions: is the data large enough to learn from, is it representative of the mail the

system will meet in real use, and is it legal and ethical to use? If the answer is no, the arrow loops back and more or better data is collected, because no algorithm can repair a dataset that does not describe the problem.

Raw data is rarely ready for learning, so the next box is cleaning and pre-processing. HTML tags are stripped, text is converted to lower case and split into words, duplicate messages are removed, missing subject lines are filled with a blank marker, character-encoding errors are corrected, and common words such as “the” and “and” may be dropped. Words are often reduced to their stems so that “winning” and “wins” are treated alike, and the labels themselves are spot-checked for mistakes, since wrongly labelled examples teach the model the wrong lesson.

3.3 Feature Extraction and Algorithm Selection

Feature extraction converts each cleaned message into numbers, because algorithms can only learn from numeric inputs. The simplest representation is a bag of words that counts how often each word occurs, and a better one is TF-IDF, in which a word’s weight is its frequency in the message multiplied by the logarithm of the total number of messages divided by the number of messages that contain it, so that rare but telling words such as “lottery” count for more than common ones. Metadata features are added as well, including the number of links, the share of capital letters, the number of exclamation marks, the presence of an attachment and the age of the sender’s domain. A feature-selection box then removes uninformative columns using chi-square scores or regularisation, and numeric features are scaled where the algorithm needs it. The data is finally split, with class proportions preserved, into about 80 per cent for training and 20 per cent for testing, and the vocabulary and scaling statistics are computed on the training portion only so that no information leaks from the test set.

Algorithm selection is drawn as a decision because the right choice depends on the type and size of the data, the need for interpretability and the available computing power. For text classification the usual candidates are Naive Bayes, which is very fast and a strong baseline but assumes that words occur independently; logistic regression, which is interpretable and works well with TF-IDF features; linear support vector machines, which excel on high-dimensional sparse text; and random forests or boosted trees, which capture non-linear feature interactions at the cost of slower training and lower interpretability. In practice the team trains a fast baseline first and moves to more complex models only if the baseline falls short.

3.4 Training and Validation

In training, the chosen algorithm is fitted to the training data: it searches for the parameters, such as regression coefficients or tree split points, that minimise a loss function measuring the gap between its predictions and the true labels. Hyperparameters, such as the regularisation strength or the depth of a tree, are set by the designer, and to choose them without touching the test set the team uses k-fold cross-validation, in which the training data is divided into k parts and the model is trained k times, each time validating on a different part. The flowchart contains a loop at this point: if both the training and validation scores are low the model is underfitting and the team improves the features or picks a more expressive algorithm, whereas if the training score is high but the validation score is much lower the model is overfitting and the team adds regularisation, simplifies the model or collects more data. Only when validation performance is acceptable does the arrow leave the loop and continue to the testing stage.

3.5 Testing, Evaluation and Deployment

Testing is the single, final measurement of the chosen model on the held-out data, which simulates the unseen mail it will meet in production. Predictions are summarised in a confusion matrix of true positives, false positives, true negatives and false negatives, from which accuracy, precision, recall and the F1-score are calculated. Table 3.1 shows illustrative figures for 1,000 test e-mails of which 400 are spam. Accuracy alone can mislead, so recall, the share of real spam that was caught, and precision, the share of flagged messages that really were spam, are examined together, and the balance between them is set by comparing the cost of a missed spam message with the far higher cost of blocking a genuine one.

Table 3.1: Metrics from an illustrative confusion matrix (TP = 380, FN = 20, FP = 9, TN = 591)

Metric	Formula	Value	Reading
Accuracy	(TP + TN) / all	97.1 %	Overall share of messages classified correctly
Precision	TP / (TP + FP)	97.7 %	Of 389 messages flagged, 380 were really spam
Recall	TP / (TP + FN)	95.0 %	Of 400 spam messages, 380 were caught
F1-score	2PR / (P + R)	96.3 %	Single balance of precision and recall

The final decision diamond asks whether the success criteria written at the start have been met. If they have, the model is packaged behind an application programming interface and monitored for data drift, because spammers keep changing their wording; if not, an arrow returns to data collection or feature extraction, showing that machine learning is an iterative cycle rather than a one-way pipeline.

MODULE 4
BACKPROPAGATION WALKTHROUGH

Question (Module 4): *Backpropagation Walkthrough: Together, step through a simple neural network's forward and backward pass calculations, showing gradient flow and weight updates.*

4.1 Introduction and Network Setup

Backpropagation is the algorithm that makes the training of neural networks practical. A network is a chain of simple mathematical operations, and training means adjusting its weights so that its output moves closer to the desired answer. Backpropagation first runs an input forward through the network to obtain a prediction, then measures the error, and finally applies the chain rule of calculus backwards from the output to every weight, revealing how much each weight contributed to the error and in which direction it should move. The group deliberately used a very small network so that every number could be checked by hand with a calculator.

The network has two input neurons, one hidden layer of two neurons and one output neuron, and every hidden and output neuron uses the sigmoid activation, sigmoid(z) = 1 / (1 + e^{-z}), whose derivative equals sigmoid(z) multiplied by (1 − sigmoid(z)). The inputs are x1 = 0.6 and x2 = 0.9 and the target is t = 1. The weights into the first hidden neuron are w1 = 0.2 and w2 = 0.4, those into the second are w3 = −0.3 and w4 = 0.5, and the shared hidden bias is b1 = 0.1. The output weights are w5 = 0.6 and w6 = −0.4, the output bias is b2 = 0.05, the loss is E = 0.5 × $(t - o)^2$ for output o, and the learning rate is 0.5.

4.2 Forward Pass and Loss

In the forward pass each neuron computes a weighted sum of its inputs plus its bias, called the net input z, and applies the activation to obtain its output. For the first hidden neuron z1 = (0.2 × 0.6) + (0.4 × 0.9) + 0.1 = 0.5800, so h1 = sigmoid(0.58) = 0.6411. For the second, z2 = (−0.3 × 0.6) + (0.5 × 0.9) + 0.1 = 0.3700, so h2 = sigmoid(0.37) = 0.5915. These outputs feed the output neuron, whose net input is z3 = (0.6 × 0.6411) + (−0.4 × 0.5915) + 0.05 = 0.1981, giving o = sigmoid(0.1981) = 0.5494. The network predicts 0.5494 when the desired answer is 1.

The loss is E = 0.5 × $(1 - 0.5494)^2$ = 0.1015, where the factor of one half is included only so that it cancels with the exponent during differentiation. To reduce this loss the network must know, for each of its eight adjustable parameters, whether increasing it would raise or lower the error. That information is the gradient, and calculating it is the purpose of the backward pass.

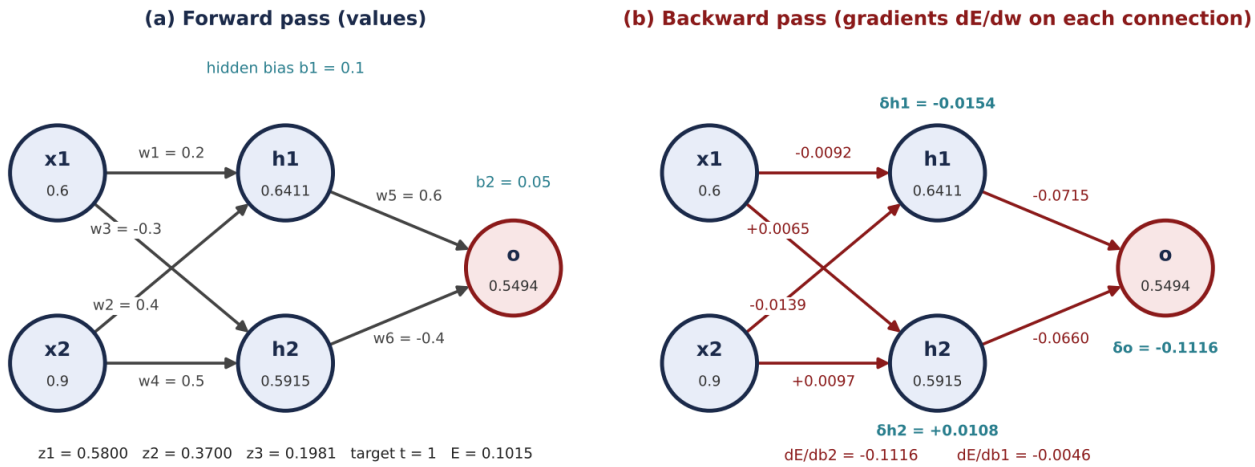


Figure 4.1: *The 2-2-1 network used in the walkthrough. (a) Values computed in the forward pass; (b) the error signals and gradients computed in the backward pass, which retrace the same connections in reverse.*

4.3 Backward Pass and Gradient Flow

The backward pass starts at the output and moves toward the input, reusing earlier results at every step. First, dE/do = o − t = 0.5494 − 1 = −0.4506, and this is multiplied by the sigmoid slope at the output, o(1 − o) = 0.5494 × 0.4506 = 0.2476, giving the output error signal δo = −0.1116. In chain-rule form dE/dw5 = (dE/do) × (do/dz3) × (dz3/dw5), and the last factor is simply h1, so each output-layer gradient is the error signal multiplied by the value carried forward: dE/dw5 = −0.1116 × 0.6411 = −0.0715,

dE/dw6 = −0.1116 × 0.5915 = −0.0660, and dE/db2 = −0.1116, since a bias is multiplied by the constant one. The negative signs show that raising these weights would lower the loss.

The error signal then flows back into the hidden layer, which is the heart of backpropagation. The share belonging to each hidden neuron depends on the connecting weight, so the first receives −0.1116 × 0.6 = −0.0669 and the second −0.1116 × (−0.4) = 0.0446. Each is multiplied by its own sigmoid slope, giving δh1 = −0.0669 × 0.6411 × (1 − 0.6411) = −0.0154 and δh2 = 0.0446 × 0.5915 × (1 − 0.5915) = 0.0108. The first-layer gradients are these deltas times the inputs: dE/dw1 = −0.0092, dE/dw2 = −0.0139, dE/dw3 = 0.0065 and dE/dw4 = 0.0097, while the shared hidden bias receives both signals, dE/db1 = −0.0154 + 0.0108 = −0.0046. Notice that w3 and w4 have positive gradients because the second hidden neuron has a negative output weight, so for them the loss falls when the weight decreases.

4.4 Weight Update and Verification

Gradient descent now moves every weight a small step against its gradient: new weight = old weight − learning rate × gradient. The output layer becomes w5 = 0.6 − 0.5 × (−0.0715) = 0.6358, w6 = −0.4 − 0.5 × (−0.0660) = −0.3670 and b2 = 0.05 − 0.5 × (−0.1116) = 0.1058, and the first layer becomes w1 = 0.2046, w2 = 0.4069, w3 = −0.3032, w4 = 0.4951 and b1 = 0.1023. All eight values are collected in Table 4.1.

Table 4.1: Gradients and updated parameters after one step

Parameter	Old value	Gradient dE/d(parameter)	New value = old − 0.5 × gradient
w1	0.2	−0.00924	0.2046
w2	0.4	−0.01386	0.4069
w3	−0.3	0.00647	−0.3032
w4	0.5	0.00970	0.4951
w5	0.6	−0.07152	0.6358
w6	−0.4	−0.06599	−0.3670
b1	0.10	−0.00462	0.1023
b2	0.05	−0.11156	0.1058

A second forward pass with the updated weights confirms that learning has occurred: the hidden outputs become h1 = 0.6437 and h2 = 0.5905, the prediction rises from 0.5494 to 0.5740, and the loss falls from 0.1015 to 0.0907, as Table 4.2 shows. One step gives only a modest improvement because the learning rate is modest, and repeating the cycle of forward pass, loss, backward pass and update over many iterations steadily drives the loss toward its minimum.

Table 4.2: Verification with a second forward pass

Quantity	Before update	After update
Hidden output h1	0.6411	0.6437
Hidden output h2	0.5915	0.5905
Network output o	0.5494	0.5740
Loss E	0.1015	0.0907

4.5 Observations

Three lessons emerged. First, the hidden-layer deltas (−0.0154 and 0.0108) are roughly an order of magnitude smaller than the output delta (−0.1116), because every step backwards multiplies the signal by a sigmoid slope that can never exceed 0.25; in deep networks this repeated shrinking causes the vanishing-gradient problem, one reason modern networks prefer ReLU activations, careful initialisation, normalisation layers and residual connections. Second, the learning rate sets the step size: too small and training is slow, too large and the updates overshoot the minimum so that the loss oscillates or diverges. Third, backpropagation is efficient because each error signal is computed once and reused, so the backward pass costs about the same as the forward pass, which is what allows networks with billions of parameters to be trained. In practice examples are processed in mini-batches whose gradients are averaged before each update, and optimisers such as momentum and Adam refine the plain gradient-descent step used here. A useful safeguard is gradient checking, in which each analytic gradient is compared with the slope estimated by nudging the weight by a tiny amount and recomputing the loss; agreement to several decimal places confirms that the hand calculation, and later any software implementation, is correct.

MODULE 5
CONTENT AI EVALUATION CHALLENGE

Question (Module 5): *Content AI Evaluation Challenge: Groups generate content using an NLP tool, then evaluate clarity, tone, and body-language cues for multimedia formats.*

5.1 Introduction and Objective

Natural language processing, or NLP, is the branch of artificial intelligence that enables computers to read, interpret and generate human language. Modern generative tools are built on transformer-based large language models, which are trained on vast collections of text to predict the next word and can therefore draft essays, e-mails and scripts on request. Because such tools produce fluent text instantly, the real challenge is no longer producing content but judging whether it is good enough to use. This challenge was designed around that skill: the group generates content with an NLP tool and then evaluates it on clarity, tone and, for multimedia formats, the body-language cues that accompany the words. The objectives were to practise writing effective prompts, to build a transparent rubric that several reviewers can apply consistently, and to understand why the quality of a text and the quality of its delivery must be assessed separately when the same message becomes a video or an avatar speech.

5.2 Content Generation Using an NLP Tool

The task adopted was a script of about one hundred words in which a university introduces a new artificial intelligence laboratory to first-year applicants, prepared for use as a web article, a spoken narration and a short video presented by a digital avatar. The language model received a structured prompt stating role, audience, purpose, length, tone and constraints, as summarised in Table 5.1. Variants can be produced by changing one element at a time, such as the tone or the audience, so that the effect of each change can be observed, and every draft is saved together with the prompt that created it.

Table 5.1: Elements of the structured prompt used in the challenge

Element	Setting used in the task
Role	Act as an admissions officer
Audience	First-year applicants
Purpose	Introduce the new artificial intelligence laboratory
Length	About one hundred words
Tone	Friendly but professional
Constraints	Avoid jargon; end with a call to action

The script generated by the language model for this prompt is reproduced below exactly as produced, and it is the text evaluated in the sections that follow.

AI-generated script (100 words) – prompt: role, audience, purpose, length, tone and constraints as in Table 5.1

“Welcome to the new Artificial Intelligence Laboratory! This is a friendly space where first-year students can explore ideas, build small projects and learn by doing. You will work with real data, train simple models and see how smart systems make decisions. Our mentors will guide you at every step, so you never need to feel lost. You do not need any experience, only curiosity and a willingness to ask questions. Come to our open day next Saturday, tour the laboratory and meet the team. Register on the university website today and take your first step into the world of AI.”

5.3 Evaluating Clarity and Tone

Clarity measures how easily the intended audience understands the message on first reading or hearing. Readability formulas such as the Flesch Reading Ease score estimate difficulty from sentence length and word length. The generated script contains 7 sentences and 100 words, an average of 14.3 words per sentence with a longest sentence of 18 words, and its Flesch Reading Ease score works out to about 63, which falls in the “plain English” band suitable for a general adult audience and for first-year students. Qualitative checks add to the numbers: the main message, that a new laboratory is open and applicants should visit it, can be stated in one sentence, each sentence carries a single idea, and the text ends with a clear call to action. Factual accuracy is verified separately against reliable sources, because generative models can produce confident but invented statements, known as hallucinations; here the open-day date and the registration page must be confirmed by the university before publication.

Tone is the attitude a text conveys, such as formal, friendly, persuasive or empathetic. Evaluators compare the tone requested in the prompt with the tone readers actually perceive, examine word choice and sentence rhythm, and check consistency from start to finish. The script is warm and encouraging through phrases such as “friendly space” and “never need to feel lost”, yet it remains professional because it makes no slang or exaggerated promises, so the perceived tone matches the requested one. Sentiment classifiers give a rough indication, but human judgement remains the final authority, because sarcasm and cultural context are hard for software to detect.

5.4 Evaluating Body-Language Cues in Multimedia

When the same text is delivered through video, a presentation or an avatar, meaning is carried by non-verbal signals as well as words, and an excellent script can be undermined by poor delivery. The rubric therefore examines facial expressions, which should match the emotional content of the words; gaze and eye contact, which should be directed at the audience or camera; gestures, which should emphasise key points and stay synchronised with speech; posture and head movement, which signal confidence; and vocal delivery, including pace, pauses, pitch and emphasis. Lip synchronisation is checked as well, since small mismatches create the uncanny-valley effect, in which an almost-human figure appears unsettling. Each cue is scored on a five-point scale with written descriptors, and at least two evaluators score independently before differences are discussed or measured with an agreement statistic such as Cohen’s kappa. Because the meaning of gestures and eye contact varies between cultures, scores are interpreted with the intended audience in mind. Table 5.2 shows the rubric in a form that reviewers can apply directly.

Table 5.2: Body-language and delivery rubric (five-point scale)

Cue	1 – absent or distracting	3 – acceptable	5 – natural and fully supportive
Facial expression	Blank or contradicts the words	Matches the words some of the time	Matches the emotional content throughout
Gaze and eye contact	Looks away; no contact	Occasional contact with camera	Directed at the audience consistently
Gestures	Absent, or repetitive and mechanical	Present but loosely timed	Emphasise key points, in step with speech
Posture and head movement	Stiff, slouched or erratic	Neutral and stable	Signals confidence and engagement
Vocal delivery	Monotone; wrong pace	Clear but flat in places	Good pace, pauses, pitch and emphasis
Lip synchronisation	Obvious mismatch	Minor lapses	Mouth movement matches the audio

5.5 Findings, Ethics and Conclusion

Combining the three dimensions gives a balanced scorecard in which clarity, tone and delivery are reported separately and then weighted according to the purpose of the content: a web article leans on clarity and tone and is not scored for delivery, whereas an avatar video gives delivery a large share of the weight. Generative tools are strongest at producing well-structured first drafts and adapting a message to different audiences, but they need human oversight for accuracy, originality and cultural sensitivity. Ethical practice forms part of the evaluation: AI assistance should be disclosed where appropriate, confidential data should never be pasted into public tools, outputs should be checked for bias and unintended plagiarism, and synthetic voices or faces must never be used to impersonate real people. Evaluating AI-generated content is thus a structured, repeatable process rather than a matter of impression.

The challenge also follows an iterative workflow. A structured prompt produces a draft; the draft is scored independently on clarity, tone and, where relevant, delivery; the scorecard is compared with the objectives set at the start; and, if any score falls short, the prompt is revised by changing one element at a time, for example by asking for shorter sentences when the readability score is too low or by requesting a warmer register when the tone is judged too formal, after which the cycle repeats. Keeping every prompt, draft and score in a log makes the process reproducible and lets the group show exactly which change produced which improvement. The group also noted the limitations of the exercise: a single short script and a small panel of reviewers cannot support statistical claims, readability formulas were designed for human-written English prose, and the body-language rubric was applied to a described rather than a filmed performance, so the results should be treated as a worked demonstration of the method and not as a benchmark of any particular tool. Future work should repeat the exercise with several language models, several prompts and a larger, more diverse panel of reviewers so that differences between tools can be measured.

5.6 Overall Conclusion and Future Scope

The four modules of this mini report follow one idea from the data upward. Module 2 showed how a big data system, here the recommendation engine of an online retailer, moves information through its sources, its ingestion and storage layers, its batch and stream processing, and its outputs, and how those outputs return to the system as fresh inputs through a feedback loop. Handling volume, velocity and variety at that scale is what makes the later stages of intelligence possible, because every learning algorithm is only as good as the data that reaches it. Module 3 turned the processing step into a disciplined machine learning workflow in which problem definition, data collection, cleaning, feature extraction, algorithm selection, training, testing and evaluation are connected by decision points that can send the team back to an earlier stage; the spam-classifier example demonstrated that the process is a cycle of improvement and not a one-way pipeline.

Module 4 opened the neural network that sits inside many such models and verified by hand that a single backpropagation step lowers the loss from 0.1015 to 0.0907, while also revealing why gradients shrink as they travel backwards through sigmoid layers and why the learning rate must be chosen with care. Module 5 addressed the newest layer, generative AI, and argued that fluent output must still be judged for clarity, tone, delivery and ethics before it is trusted. Taken together, the modules describe a complete journey: data is collected at scale, processed by learning algorithms, represented inside neural networks, and finally used by generative models to produce content that people read, hear and watch. Working on them as a group also built practical skills in drawing flowcharts, checking numerical calculations, designing rubrics, dividing work and presenting technical ideas in clear language.

Table 5.3: Summary of the four modules

Module	Core activity	Key takeaway
2 Big Data Process Mapping	Mapped the data flow of a recommendation system	Choose batch, streaming, storage and cache layers to match each type of data
3 ML Process Flow	Drew a complete flowchart for a spam classifier	Machine learning is iterative; decision diamonds feed improvements back
4 Backpropagation Walkthrough	Computed forward and backward passes by hand	One update lowered the loss from 0.1015 to 0.0907; gradients shrink with depth
5 Content AI Evaluation	Generated and scored AI content and its delivery	Judge clarity, tone, delivery and ethics separately, then combine them

Beyond the technical findings, the exercise produced clear learning outcomes. The group learned to describe a real system in terms of its data flow instead of its user interface, to translate a verbal project description into a standard flowchart whose symbols have precise meanings, to carry out a full neural-network calculation by hand and check each intermediate value, and to design an evaluation rubric whose scores can be defended and repeated by other reviewers. It also became clear that the four topics are not separate subjects but successive layers of one pipeline: without large and well-managed data there is nothing to learn from; without a sound machine learning process the learning cannot be trusted; without an understanding of how gradients train a network the behaviour of modern models remains a black box; and without careful evaluation the fluent output of generative systems can mislead as easily as it can help. Responsible practice therefore requires attention to privacy and consent when data is collected, to bias and overfitting when models are trained, to transparency when results are explained, and to disclosure and human review when generated content is published.

Several extensions are possible in future work. The big data map could be repeated for a second system, such as smart-city sensors or a navigation service, and the two compared on the five Vs, or a small version of the recommender could be implemented with Apache Spark on a public dataset such as MovieLens. The spam flowchart could be implemented end to end with scikit-learn on a public corpus, with the metrics in Table 3.1 measured on real test data instead of illustrated, and with a comparison of Naive Bayes, logistic regression and boosted trees. The backpropagation example could be reproduced in PyTorch or TensorFlow to confirm the hand calculation against automatic differentiation, and extended to deeper networks, ReLU activations and mini-batch training with the Adam optimiser. Finally, the Module 5 rubric could be applied by several reviewers to drafts from different language models and different prompts, so that inter-rater agreement can be reported with Cohen’s kappa, and the study could be widened to retrieval-augmented generation, multilingual content and watermarking of AI-generated media. Each of these extensions would move the work from demonstration toward evidence, and each reinforces the central lesson of the report: that the practical value of big data, machine learning, neural networks and generative AI depends on careful engineering, honest evaluation and responsible use. A further extension would be to package the four modules as one end-to-end demonstration, in which logged data from a small web application feeds a trained classifier whose predictions are explained to users through generated text, and the quality of that text is scored with the same rubric.

5.7 References

1. N. Marz and J. Warren, *Big Data: Principles and Best Practices of Scalable Real-Time Data Systems*. Manning Publications, 2015.
2. V. Mayer-Schönberger and K. Cukier, *Big Data: A Revolution That Will Transform How We Live, Work, and Think*. Houghton Mifflin Harcourt, 2013.
3. J. Dean and S. Ghemawat, "MapReduce: Simplified Data Processing on Large Clusters," *Communications of the ACM*, vol. 51, no. 1, 2008.
4. M. Zaharia et al., "Apache Spark: A Unified Engine for Big Data Processing," *Communications of the ACM*, vol. 59, no. 11, 2016.
5. J. Kreps, N. Narkhede and J. Rao, "Kafka: A Distributed Messaging System for Log Processing," *Proc. NetDB Workshop*, 2011.
6. G. Linden, B. Smith and J. York, "Amazon.com Recommendations: Item-to-Item Collaborative Filtering," *IEEE Internet Computing*, vol. 7, no. 1, 2003.
7. B. Smith and G. Linden, "Two Decades of Recommender Systems at Amazon.com," *IEEE Internet Computing*, vol. 21, no. 3, 2017.
8. Y. Koren, R. Bell and C. Volinsky, "Matrix Factorization Techniques for Recommender Systems," *Computer*, vol. 42, no. 8, 2009.
9. A. Géron, *Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow*, 3rd ed. O'Reilly Media, 2022.
10. F. Pedregosa et al., "Scikit-learn: Machine Learning in Python," *Journal of Machine Learning Research*, vol. 12, 2011.
11. T. Hastie, R. Tibshirani and J. Friedman, *The Elements of Statistical Learning*, 2nd ed. Springer, 2009.
12. C. M. Bishop, *Pattern Recognition and Machine Learning*. Springer, 2006.
13. M. Kuhn and K. Johnson, *Applied Predictive Modeling*. Springer, 2013.
14. I. Goodfellow, Y. Bengio and A. Courville, *Deep Learning*. MIT Press, 2016.
15. D. E. Rumelhart, G. E. Hinton and R. J. Williams, "Learning Representations by Back-Propagating Errors," *Nature*, vol. 323, 1986.
16. S. Hochreiter, "The Vanishing Gradient Problem During Learning Recurrent Neural Nets and Problem Solutions," *Int. J. Uncertainty, Fuzziness and Knowledge-Based Systems*, vol. 6, no. 2, 1998.
17. D. P. Kingma and J. Ba, "Adam: A Method for Stochastic Optimization," *Proc. ICLR*, 2015.
18. A. Vaswani et al., "Attention Is All You Need," *Advances in Neural Information Processing Systems*, 2017.
19. T. B. Brown et al., "Language Models Are Few-Shot Learners," *Advances in Neural Information Processing Systems*, 2020.
20. R. Bommasani et al., "On the Opportunities and Risks of Foundation Models," Stanford University, 2021.
21. E. M. Bender, T. Gebru, A. McMillan-Major and S. Shmitchell, "On the Dangers of Stochastic Parrots: Can Language Models Be Too Big?" *Proc. ACM FAccT*, 2021.
22. Z. Ji et al., "Survey of Hallucination in Natural Language Generation," *ACM Computing Surveys*, vol. 55, no. 12, 2023.
23. R. Flesch, "A New Readability Yardstick," *Journal of Applied Psychology*, vol. 32, no. 3, 1948.
24. J. Cohen, "A Coefficient of Agreement for Nominal Scales," *Educational and Psychological Measurement*, vol. 20, no. 1, 1960.
25. M. Mori, K. F. MacDorman and N. Kageki, "The Uncanny Valley," *IEEE Robotics & Automation Magazine*, vol. 19, no. 2, 2012.
26. Apache Software Foundation, "Apache Hadoop." Available: https://hadoop.apache.org/